

BakeryOS: un ERP para la gestión de panaderías

Diseño, arquitectura y desarrollo de un ecosistema digital avanzado en la nube para la optimización de procesos de producción, stock, facturación y control por lenguaje natural en panaderías artesanales

AUTOR:

Noah Catalán Rosell

TUTOR DEL PROYECTO:

Miquel Antoni Capellà Arrom

ENTORNO / DESPLIEGUE:

Vercel & Google Firebase Cloud

FECHA DE PRESENTACIÓN:

Junio 2026

ÍNDICE DE SECCIONES

1. Introducción y Contexto	3
2. Impacto y Aspectos Éticos	4
3. Metodología y Herramientas	5
4. Planificación del Proyecto	6
5. Alcance y Funcionalidades del Sistema	7
6. Entorno de Trabajo y Prototipado	8
7. Desarrollo de Sprints 1 y 2	9
8. Desarrollo de Sprint 3: Arquitectura y Producción Automatizada	10
9. Desarrollo de Sprint 4: BakeryAI NLP y Sistema de Notificaciones	12
10. Conclusiones y Trabajo Futuro	14

1. INTRODUCCIÓN Y CONTEXTO

1.1. Problemática y Oportunidad

La inspiración para este proyecto nace de la observación directa y la experiencia personal. Como cliente habitual durante años de la panadería de mi barrio, he podido constatar las graves dificultades operativas a las que se enfrentan diariamente. A pesar de ofrecer un producto artesanal de altísima calidad, muchos de estos pequeños negocios todavía dependen hoy en día de métodos de gestión completamente manuales, tales como libretas de papel, pizarras de tiza y hojas de cálculo locales dispersas para procesos que resultan críticos para la viabilidad económica del negocio.

Esta dependencia de métodos anticuados genera ineficiencias sustanciales que restringen su rentabilidad y limitan su potencial de crecimiento en un mercado cada vez más competitivo. La problemática principal del sector se sintetiza en tres vertientes fundamentales:

- **Control de inventario y desperdicio de producción:** La falta de un sistema centralizado y digital de gestión permite que se produzcan mermas constantes debido a excesos de producción o a compras desorganizadas que no se corresponden con las necesidades reales de stock diario.
- **Cálculo inexacto de costes:** Sin una herramienta capaz de calcular el coste de las recetas (escandallo) de manera automática basada en el precio actualizado de compra de los ingredientes, resulta extremadamente complejo para los panaderos conocer el coste real de cada producto y fijar un precio de venta estratégico que garantice un margen de beneficio constante.
- **Gestión administrativa y de pedidos recurrentes:** Tareas como el seguimiento de pedidos para clientes de hostelería habituales o la emisión de facturas mensuales representan un proceso de administración tedioso y altamente propenso a errores humanos que restan valioso tiempo a la labor artesanal de panificación.

Ante este escenario, se detectó una oportunidad clara para conceptualizar y desarrollar un software de tipo ERP (Enterprise Resource Planning) extremadamente intuitivo, centrado en la facilidad de uso y diseñado específicamente para eliminar la barrera tecnológica en pequeños negocios artesanales de panadería.

1.2. Objetivos del Proyecto

El objetivo principal de BakeryOS es desarrollar una aplicación web funcional, robusta y accesible que permita a las panaderías artesanales digitalizar de forma integral y centralizada la gestión de su inventario, planificación de producción, gestión de recetas y relación de pedidos y facturación con clientes B2B/B2C.

Para lograrlo, se han trazado los siguientes objetivos específicos:

- **Implementar un módulo de inventario integral:** Diseñar y maquetar una interfaz ágil de control de materias primas y productos elaborados, con estados visuales semafóricos ("ok", "bajo", "alerta") e históricos dinámicos sincronizados en tiempo real.
- **Crear un sistema de recetas (escandallos) y planificación de producción:** Desarrollar la lógica de base de datos que relacione los ingredientes con los productos terminados, determinando de forma automática el coste total de producción y permitiendo generar órdenes diarias de horneado.
- **Desarrollar un módulo transaccional de clientes y pedidos:** Dotar a la aplicación de un registro de clientes B2B (cafeterías, restaurantes) y un motor de planificación que permita configurar pedidos recurrentes diarios, automatizando el flujo administrativo tradicional.
- **Diseñar una interfaz de usuario minimalista y responsiva:** Aplicar principios avanzados de Experiencia de Usuario (UX) a través de un diseño visual limpio, elegante y adaptable a dispositivos móviles y tabletas utilizados a pie de obrador.
- **Diseñar e implementar una interfaz de control por lenguaje natural (BakeryAI):** Crear un módulo innovador que permita a los operarios interactuar con el ERP a través de voz y comandos textuales naturales en español (ej. *"añadir 20 a Harina de Fuerza T80"*), facilitando el trabajo con las manos ocupadas en la masa.

2. IMPACTO Y ASPECTOS ÉTICOS

2.1. Sostenibilidad y Accesibilidad

Sostenibilidad Ambiental: BakeryOS aporta un impacto ecológico sumamente positivo al obrador. Al digitalizar por completo el flujo de albaranes, pedidos y la facturación, se reduce prácticamente a cero el consumo de papel físico utilizado habitualmente en las comandas diarias. Asimismo, al planificar con exactitud la producción basada en los pedidos consolidados del día y el stock real de ingredientes, se reduce de manera directa el desperdicio alimentario, lo que se alinea estrechamente con el Objetivo de Desarrollo Sostenible (ODS) 12 de la Agenda 2030 de la ONU.

Accesibilidad Universal: Conscientes de que el personal del obrador no posee un perfil tecnológico avanzado y realiza sus labores bajo condiciones de trabajo físicamente exigentes, la accesibilidad es un factor central de BakeryOS. La interfaz utiliza HTML semántico, contrastes cromáticos de color de nivel AAA y fuentes tipográficas limpias que evitan la fatiga visual. La incorporación del asistente de voz BakeryAI supone un salto radical en materia de accesibilidad al permitir que personas con limitaciones de motricidad fina o manos sucias puedan dictar y ejecutar tareas sin necesidad de tocar la pantalla táctil de una tableta.

2.2. Usabilidad y Diversidad

La aplicación ha sido concebida bajo la máxima de "el software debe adaptarse al usuario y no al revés". En lugar de estructurar un ERP saturado de campos numéricos y menús infinitos, la interfaz presenta tableros limpios donde la información clave resalta gracias al uso de micro-animaciones no intrusivas y colores de significado unívoco (rojo para urgencias de stock, azul para procesos activos, verde para tareas completadas). La usabilidad ha sido corroborada mediante pruebas empíricas imitando las dinámicas reales de una panadería típica, optimizando de esta manera los flujos de clics a su mínima expresión.

2.3. Protección de Datos y Licenciamiento

El sistema gestiona datos sensibles correspondientes a cuentas de clientes B2B, cifras de facturación, direcciones de correo y números telefónicos. De acuerdo con el Reglamento

General de Protección de Datos (RGPD), se han adoptado las siguientes medidas técnicas de seguridad en el entorno de la nube de Google Firebase:

- **Autenticación robusta:** La gestión de accesos se realiza a través de Firebase Authentication, delegando en los servidores de Google el cifrado y salting de contraseñas de los usuarios mediante algoritmos criptográficos robustos de estándar industrial.
- **Reglas de Seguridad Granulares:** Se han configurado detalladamente las reglas de Firebase Firestore (`firestore.rules`) para validar que las operaciones de lectura y escritura sobre los documentos estén restringidas únicamente a usuarios autenticados que sean propietarios del respectivo espacio de datos, bloqueando cualquier intento de consulta cross-tenant.
- **Protección contra ataques XSS y CSRF:** El frontend en Next.js cuenta con protección contra inyección de código al escapar automáticamente los inputs en el renderizado de React, minimizando los riesgos tradicionales en el intercambio de datos dinámicos.

3. METODOLOGÍA Y HERRAMIENTAS

3.1. Metodología de Desarrollo

Para la ejecución del proyecto de manera organizada y sistemática, se ha adoptado una metodología ágil basada en **Scrum**, dividiendo el ciclo de vida del desarrollo en sprints estructurados de dos semanas de duración. La visibilidad de los entregables y la distribución de las tareas se gestiona de forma profesional mediante un tablero Kanban digital en la plataforma Trello, permitiendo rastrear el estado real del trabajo e iterar rápidamente ante la aparición de refinamientos o nuevos requisitos de negocio.

3.2. Arquitectura Tecnológica y Stack de Software

La arquitectura tecnológica se caracteriza por un enfoque puramente "serverless", moderno y tipado estrictamente de extremo a extremo:

TECNOLOGÍA	ROL EN LA ARQUITECTURA	JUSTIFICACIÓN TÉCNICA
Next.js 16 (App Router)	Frontend & Server Rendering	Ofrece una navegación ultra-rápida, optimización nativa de imágenes y enrutamiento basado en carpetas sumamente escalable.
React 19 & TypeScript	Componentes e Interfaces de datos	Garantiza tipado estático riguroso que previene fallos numéricos en coma flotante y facilita la escalabilidad del código.
Firebase Auth & Firestore	Backend-as-a-Service (BaaS)	Servicio de base de datos NoSQL de bajísima latencia con sincronización en tiempo real y flujo de autenticación seguro e instantáneo.
Tailwind CSS v4	Framework de Estilos Visuales	Permite un diseño visual premium y fluido mediante utility classes de alto rendimiento CSS sin código redundante.
Web Speech API (Nativa)	Reconocimiento y Síntesis de voz	Habilita el motor de interacción por voz de BakeryAI sin costes adicionales ni latencias de APIs de terceros.

Nota de Arquitectura: El stack "serverless" adoptado elimina por completo la necesidad de mantener servidores locales o instancias virtuales encendidas continuamente, lo que reduce a cero los costes fijos de mantenimiento de infraestructura para los clientes de BakeryOS.

4. PLANIFICACIÓN DEL PROYECTO

4.1. Fases del Proyecto y Cronograma

El proyecto ha sido estructurado temporalmente en cinco fases críticas, distribuidas de forma que los sprints iniciales consoliden las bases funcionales e infraestructuras, mientras que los sprints avanzados se enfoquen en la integración, usabilidad superior y optimizaciones técnicas:

FASE / SPRINT	OBJETIVOS CLAVE	ENTREGABLES PRINCIPALES	FECHAS DE EJECUCIÓN
Sprint 1: Core & Auth	Inicialización y Base de Datos	Configuración Next.js/Firebase, CRUD completo de Inventario y Control de Accesos de usuarios.	Enero 2026
Sprint 2: UX & Refinamiento	Estética Superior y Modo Oscuro	Carga de imágenes de perfil en Firebase Storage, corrección de decimales y steppers en inputs.	Febrero - Marzo 2026
Sprint 3: Producción & DB Layer	Lógica de Obrador y Robustez	Estandarización del patrón Repository, transacciones atómicas Firestore en la producción diaria.	Marzo - Abril 2026
Sprint 4: BakeryAI & Notificaciones	IA Conversacional y Premium UI	Desarrollo del motor NLP de BakeryAI, control de voz y Contexto Global de Notificaciones (Toasts).	Abril - Mayo 2026
Fase 5: Cierre & Defensa	Estabilización y Documentación	Generación de la memoria final PDF mediante WeasyPrint y preparación de la presentación de defensa.	Junio 2026

5. ALCANCE Y FUNCIONALIDADES DEL SISTEMA

5.1. Funcionalidades Principales por Módulo

El sistema BakeryOS se articula en torno a los siguientes bloques funcionales:

- **Módulo de Dashboard Integrado:** Muestra indicadores de rendimiento (KPIs) clave en tiempo real, incluyendo el total de producción pendiente para el día, número de pedidos a despachar y alertas inmediatas sobre ingredientes con stock en estado crítico.
- **Módulo de Inventario Avanzado:** Permite gestionar el alta, edición y eliminación de ingredientes y productos elaborados. El inventario cuenta con filtros inteligentes y estados visuales basados en el stock mínimo fijado por el panadero.
- **Módulo de Fórmulas y Obrador (Producción):** Gestión de los escandallos asociados a cada producto de venta. Habilita la planificación diaria de horneados estimando costes teóricos y tiempos de producción.
- **Módulo CRM (Clientes y Pedidos de Venta):** Facilita el control de clientes, el registro de pedidos puntuales y la programación de pedidos periódicos de distribución continua a cafeterías y comercios asociados.
- **Módulo de Facturación Automatizada:** Agrupa pedidos completados de un mismo cliente para generar facturas formalizadas en PDF con cálculo automático de impuestos correspondientes (IVA).
- **Asistente Virtual BakeryAI:** Habilita la interacción a través de reconocimiento de voz y comandos textuales, procesando órdenes de actualización de inventario e informes de stock de forma verbal e interactiva.

6. ENTORNO DE TRABAJO Y PROTOTIPADO

6.1. Enlaces a los Entornos de Trabajo

Para garantizar el rigor académico y facilitar el control docente, los entornos reales de BakeryOS están disponibles y actualizados continuamente en las siguientes direcciones de producción y desarrollo:

- **Repositorio Oficial de Código en GitHub:** <https://github.com/noah-catalan/BakeryOS>
- **Tablero Ágil de Planificación en Trello:** [Tablero Kanban BakeryOS](#)
- **Despliegue en Producción (Vercel):** <https://bakery-os-ten.vercel.app/>
- **Credenciales del Entorno de Demostración:**
 - **Usuario Administrador:** admin@bakeryos.com
 - **Contraseña de Acceso:** admin123

6.2. Metodología de Flujo de Trabajo en Git (Git Flow Simplificado)

A nivel de control de versiones se ha seguido una rama main permanentemente enlazada con Vercel para despliegue continuo (CI/CD), utilizando ramas de características temporales (feature/nombre-de-la-mejora) para aislar los cambios del código en desarrollo, garantizando que el entorno público de producción permanezca siempre funcional y libre de regresiones.

7. DESARROLLO DE SPRINTS 1 Y 2

7.1. Sprint 1: Construcción del Core e Infraestructura Base

Durante el Sprint 1 se sentaron los cimientos de BakeryOS. Se inicializó la aplicación en Next.js estructurando el enrutador basado en `src/app`. Se conectó con el proyecto Firebase y se configuró Firebase Auth para bloquear las rutas del ERP, redirigiendo a los usuarios no autenticados a la vista de Login.

Adicionalmente, se desarrolló el CRUD de inventario en tiempo real sobre Firebase Firestore, permitiendo dar de alta materias primas que inmediatamente se actualizan en los clientes conectados. Para validar la efectividad de la base de datos NoSQL, se programó un script de generación de datos semilla (Seeding) capaz de rellenar la base de datos con ingredientes básicos, recetas preconfiguradas y clientes ficticios con un solo clic.

7.2. Sprint 2: Estandarización Visual, Control Numérico y Modo Oscuro

En esta etapa se refinó sustancialmente la experiencia del usuario (UX) para dar al software un acabado profesional. Se implementó un motor dinámico de cambio de temática mediante variables CSS que permite alternar entre modo claro y oscuro de manera impecable, guardando la preferencia del usuario en el `localStorage` del navegador.

Para dar soporte al obrador en la carga de documentos e información personal, se integró el almacenamiento de imágenes de perfil directamente conectadas con Firebase Storage. Por último, se corrigió un problema crítico de imprecisión en los cálculos decimales mediante el formateo numérico con `toFixed(2)` y se ajustó la lógica en los steppers incrementales de cantidad para respetar siempre los stocks mínimos de cada ingrediente, impidiendo el registro inadvertido de mermas ficticias o valores erróneos.

8. DESARROLLO DE SPRINT 3: ARQUITECTURA Y PRODUCCIÓN AUTOMATIZADA

8.1. Problemática Inicial y Necesidad de Refactorización

Inicialmente, BakeryOS realizaba llamadas directas a las funciones del SDK de Firebase Firestore (como `addDoc`, `updateDoc`, o `deleteDoc`) dispersas en el código de múltiples vistas y componentes React. Este diseño, aunque sencillo en etapas iniciales, generaba un fuerte acoplamiento a Firebase y dificultaba enormemente la mantenibilidad y la implementación de pruebas unitarias. Asimismo, las operaciones complejas (como deducir ingredientes del inventario al finalizar una orden de producción) requerían orquestar múltiples lecturas y escrituras consecutivas, exponiendo al sistema a condiciones de carrera o inconsistencias de datos en caso de cortes de red.

8.2. Solución Aplicada: Patrón Repository (Capa de Servicios Decoplada)

Para subsanar esta debilidad de arquitectura, se implementó el **Patrón Repository** mediante una capa de servicio centralizada en `src/lib/services/db.ts`. Esta clase aislada, denominada `dbService`, actúa como un punto único de interacción con la persistencia de datos. De esta forma, las páginas y componentes del frontend consumen métodos de alto nivel con nombres de dominio semánticos, desconociendo por completo la lógica interna de almacenamiento de Firestore.

A continuación se detalla la implementación técnica de la interfaz genérica y del servicio de base de datos consolidado para BakeryOS:

```

// src/lib/services/db.ts
import { db } from "../firebase";
import {
  collection, doc, addDoc, updateDoc, deleteDoc,
  getDocs, getDoc, query, where, writeBatch
} from "firebase/firestore";

class DatabaseService {
  // --- INGREDIENTES REPOSITORY ---
  async getIngredients(userId: string) {
    const q = query(collection(db, "ingredientes"), where("userId", "==", userId));
    const snapshot = await getDocs(q);
    return snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));
  }

  async saveIngredient(id: string | null, data: any) {
    if (id) {
      await updateDoc(doc(db, "ingredientes", id), data);
    } else {
      await addDoc(collection(db, "ingredientes"), data);
    }
  }

  // --- TRANSACCIÓN COMPLEJA DE PRODUCCIÓN (BATCH WRITES) ---
  async completeProductionOrder(orderId: string, recipe: any, factor: number, ingredientsDb: any[]) {
    const batch = writeBatch(db);

    // 1. Recorrer ingredientes de la receta y deducir del stock actual
    for (const ingReq of recipe.ingredientes_necesarios) {
      const dbIng = ingredientsDb.find(i => i.id === ingReq.ingredienteId);
      if (dbIng) {
        const totalConsumido = ingReq.cantidad * factor;
        const nuevoStock = Math.max(0, dbIng.stockActual - totalConsumido);

        // Recálculo del estado del ingrediente basado en stock mínimo
        let nuevoEstado = dbIng.estado;
        if (nuevoStock <= dbIng.stockMinimo / 2) nuevoEstado = 'alerta';
        else if (nuevoStock <= dbIng.stockMinimo) nuevoEstado = 'bajo';
        else nuevoEstado = 'ok';

        batch.update(doc(db, "ingredientes", dbIng.id), {
          stockActual: nuevoStock,
          estado: nuevoEstado
        });
      }
    }

    // 2. Marcar la orden de producción como completada
    batch.update(doc(db, "ordenesProduccion", orderId), {
      estado: 'completada',
      fechaCompletada: Date.now()
    });

    // 3. Confirmar la transacción atómica
    await batch.commit();
  }
}

```

```
export const dbService = new DatabaseService();
```

8.3. Gestión Atómica de Inventario en Órdenes de Producción

El mayor logro de este sprint reside en la automatización del proceso de producción diaria. Cuando un panadero marca una orden de producción como "Completada", el software calcula instantáneamente el consumo real de ingredientes multiplicando las proporciones de la receta por las tandas (lotes) producidas.

Gracias al uso de `writeBatch(db)` expuesto en nuestro servicio desacoplado, todas las actualizaciones de stock de los ingredientes y el cambio de estado de la orden de producción se ejecutan de manera **atómica** en una sola llamada de red. Esto asegura que o bien se procesan todas las deducciones de stock de forma consistente, o bien la orden no cambia de estado en absoluto en caso de fallo, protegiendo al obrador de descuadres contables accidentales.

9. DESARROLLO DE SPRINT 4: BAKERYAI NLP Y SISTEMA DE NOTIFICACIONES

9.1. BakeryAI: Motor de Procesamiento de Lenguaje Natural en Obrador

Un problema clásico en los sistemas ERP tradicionales es la fricción en la entrada de datos. Para un panadero profesional que está elaborando masa a mano, interactuar continuamente con pantallas táctiles, teclados y menús anidados resulta inviable y poco higiénico.

Para solucionar este escollo, el Sprint 4 introdujo el módulo **BakeryAI** en `src/app/asistente/page.tsx`. Este motor combina el reconocimiento de voz nativo en español del navegador (Web Speech API) con un procesador léxico e interpretativo (NLP Parser) programado específicamente para interactuar con la persistencia de datos del ERP.

El parser analiza comandos conversacionales estructurados de manera semántica, traduciéndolos a acciones directas en la base de datos Firestore y mostrando de forma inmediata "Action Cards" con la respuesta. Los comandos admitidos incluyen:

- **Gestión de Inventario:** *"añadir 20 a Harina de Fuerza T80" o "restar 5 a Azúcar Blanco"*. El sistema localiza el ingrediente, actualiza su valor acumulado de stock y recalcula sus rangos de alerta.
- **Consultas Rápidas:** *"¿Qué ingredientes están bajo mínimos?" o "mostrar alertas"*. BakeryAI filtra los ingredientes que se encuentran en estado 'bajo' o 'alerta' y los presenta en un carrusel interactivo en pantalla.
- **Creación de Ítems:** *"crear ingrediente Harina de Centeno en Harinas con stock mínimo 10"*. El parser extrae la categoría, nombre y límites para dar de alta el documento en Firestore en tiempo real.


```
// Fragmento del NLP Parser de BakeryAI (src/app/asistente/page.tsx)
const processCommand = async (text: string) => {
  const normalized = text.toLowerCase().trim();

  // Expresión regular para adición y sustracción de stock
  const stockMatch = normalized.match(/(añadir|sumar|restar|quitar)\s+(\d+)\s+a\s+(.+)/);
  if (stockMatch) {
    const action = stockMatch[1];
    const qty = parseInt(stockMatch[2]);
    const ingredientName = stockMatch[3].trim();

    // Búsqueda del ingrediente en base de datos
    const ing = findIngredientByName(ingredientName);
    if (ing) {
      const currentQty = ing.stockActual;
      const newQty = (action === 'añadir' || action === 'sumar') ? currentQty + qty : Math.max(0, currentQty - qty);

      await dbService.updateIngredientStock(ing.id, newQty);
      return { success: true, message: `Stock de "${ing.nombre}" actualizado a ${newQty} kg.` };
    }
  }

  // Comando de consulta de alertas de stock
  if (normalized.includes("bajo mínimos") || normalized.includes("alerta")) {
    const alerts = getIngredientsInAlert();
    return { success: true, data: alerts, type: 'alertList' };
  }

  return { success: false, message: "No he podido comprender la orden. Inténtalo de nuevo." };
};
```

9.2. Sistema de Notificaciones Toasts de Diseño Premium (Glassmorphic)

Originalmente, la interfaz de BakeryOS arrojaba alertas nativas del navegador (`alert()`), las cuales interrumpían drásticamente la navegación de los usuarios, bloqueaban la ejecución del hilo principal de Javascript y presentaban un diseño amateur y tosco.

Para dotar a la plataforma de un acabado verdaderamente profesional y consistente, se construyó e integró un sistema de notificaciones global basado en el patrón React Provider (`src/context/ToastContext.tsx`). Este framework proporciona un hook reutilizable (`useToast`) que permite lanzar alertas contextuales no intrusivas:

MÉTODO DEL HOOK	TIPO DE ALERTA	USO RECOMENDADO EN BAKERYOS
<code>toast.success("...")</code>	Confirmación (Verde)	Confirmar la creación exitosa de recetas, guardado de clientes y finalización de horneados.
<code>toast.error("...")</code>	Error crítico (Rojo)	Informar fallos de red con Firestore, errores al procesar facturas o validación fallida.
<code>toast.warning("...")</code>	Advertencia (Amarillo)	Alertar que una orden de producción consume más stock del que se dispone en el obrador.
<code>toast.info("...")</code>	Información (Azul)	Notificaciones de BakeryAI relativas a la activación del motor de escucha por voz.

Las tarjetas visuales de notificación han sido maquetadas con un estilo premium e hipermoderno empleando **glassmorphism** (fondos semi-translúcidos y difuminado de fondo `backdrop-filter: blur(10px)`) y equipadas con transiciones CSS de desplazamiento lateral (fade-in/fade-out) y una barra de progreso que indica el temporizador de auto-descarte (4 segundos), transformando radicalmente la UX de la herramienta.

10. CONCLUSIONES Y TRABAJO FUTURO

10.1. Evaluación del Proyecto y Objetivos Logrados

BakeryOS ha culminado su desarrollo consolidándose como un ERP altamente especializado que satisface de forma excepcional los requisitos iniciales del proyecto. La digitalización integral del flujo operativo ha permitido sustituir las herramientas manuales tradicionales por un entorno de datos en tiempo real de bajísima fricción. La implantación de la arquitectura de servicios desacoplada (Repository Pattern) dota a la aplicación de bases firmes para facilitar su mantenimiento y evolución técnica futura.

Asimismo, la culminación exitosa del asistente conversacional **BakeryAI** y la renovación de la interfaz visual de alertas demuestran que es posible aproximar la inteligencia artificial aplicada y las técnicas de usabilidad de alta gama a sectores artesanales que habitualmente se encuentran rezagados en el proceso de transformación digital.

10.2. Trabajo Futuro y Líneas de Expansión

Como líneas de trabajo futuras para la escalabilidad comercial y técnica de BakeryOS, se proponen los siguientes módulos de expansión:

- **Integración con LLMs Conversacionales Avanzados:** Migrar el motor NLP simplificado a un asistente impulsado por modelos más complejos (como Gemini Nano o APIs serverless de lenguaje), capacitando a BakeryAI para contestar consultas predictivas de ventas de gran complejidad (ej: *"¿Qué cantidades de harina deberé comprar para los pedidos del próximo viernes basándote en la previsión del tiempo?"*).
- **Módulo de Trazabilidad por Lotes:** Incorporar el seguimiento integral de lotes de materias primas con fechas de caducidad automatizadas, alertando al panadero de la proximidad del vencimiento de harinas y levaduras específicas.
- **Conexión con Balanzas y Maquinaria IoT:** Habilitar la sincronización directa mediante Bluetooth con básculas de obrador para volcar el pesaje real de los ingredientes al escandallo de la receta de forma completamente automatizada.